

虽然很大程度上可以按照自己的意愿自由地设定程序的格式，但我们所做的选择会影响程序的可读性。例如，我们可以将整个 `main` 函数写在很长的单行内，虽然这样是合乎语法的，但会非常难读。

关于 C/C++ 的正确格式的辩论是无休止的。我们的信条是，不存在唯一正确的风格，但保持一致性是非常重要的。例如，大多数程序员都对程序的组成部分设置恰当的缩进，就像我们在之前的例子中对 `main` 函数中的语句和循环体所做的那样。对于作为函数界定符的花括号，我们习惯将其放在单独一行中。我们还习惯对复合 `IO` 表达式设置缩进，以使输入输出运算符排列整齐。其他一些缩进约定也都会令越来越复杂的程序更加清晰易读。

我们要牢记一件重要的事情：其他可能的程序格式总是存在的。当你要选择一种格式风格时，思考一下它会对程序的可读性和易理解性有什么影响，而一旦选择了一种风格，就要坚持使用。

1.5 类简介

在解决书店程序之前，我们还需要了解的唯一一个 C++ 特性，就是如何定义一个数据结构（`data structure`）来表示销售数据。在 C++ 中，我们通过定义一个类（`class`）来定义自己的数据结构。一个类定义了一个类型，以及与其关联的一组操作。类机制是 C++ 最重要的特性之一。实际上，C++ 最初的一个设计焦点就是能定义使用上像内置类型一样自然的类类型（`class type`）。

在本节中，我们将介绍一个在编写书店程序中会用到的简单的类。当我们在后续章节中学习了更多关于类型、表达式、语句和函数的知识后，会真正实现这个类。

为了使用类，我们需要了解三件事情：

- 类名是什么？
- 它是在哪里定义的？
- 它支持什么操作？

对于书店程序来说，我们假定类名为 `Sales_item`，头文件 `Sales_item.h` 中已经定义了这个类。

如前所见，为了使用标准库设施，我们必须包含相关的头文件。类似的，我们也需要使用头文件来访问为自己的应用程序所定义的类。习惯上，头文件根据其中定义的类的名字来命名。我们通常使用 `.h` 作为头文件的后缀，但也有一些程序员习惯 `.h`、`.hpp` 或 `.hxx`。标准库头文件通常不带后缀。编译器一般不关心头文件名的形式，但有的 IDE 对此有特定要求。◀ 20

1.5.1 `Sales_item` 类

`Sales_item` 类的作用是表示一本书的总销售额、售出册数和平均售价。我们现在不关心这些数据如何存储、如何计算。为了使用一个类，我们不必关心它是如何实现的，只需知道类对象可以执行什么操作。

每个类实际上都定义了一个新的类型，其类型名就是类名。因此，我们的 `Sales_item` 类定义了一个名为 `Sales_item` 的类型。与内置类型一样，我们可以定义类类型的变量。当我们写下如下语句

```
Sales_item item;
```

是想表达 `item` 是一个 `Sales_item` 类型的对象。我们通常将“一个 `Sales_item` 类型的对象”简单说成“一个 `Sales_item` 对象”，或更简单的“一个 `Sales_item`”。

除了可以定义 `Sales_item` 类型的变量之外，我们还可以：

- 调用一个名为 `isbn` 的函数从一个 `Sales_item` 对象中提取 ISBN 书号。
- 用输入运算符 (`>>`) 和输出运算符 (`<<`) 读、写 `Sales_item` 类型的对象。
- 用赋值运算符 (`=`) 将一个 `Sales_item` 对象的值赋予另一个 `Sales_item` 对象。
- 用加法运算符 (`+`) 将两个 `Sales_item` 对象相加。两个对象必须表示同一本书 (相同的 ISBN)。加法结果是一个新的 `Sales_item` 对象，其 ISBN 与两个运算对象相同，而其总销售额和售出册数则是两个运算对象的对应值之和。
- 使用复合赋值运算符 (`+=`) 将一个 `Sales_item` 对象加到另一个对象上。

关键概念：类定义了行为

当你读这些程序时，一件要牢记的重要事情是，类 `Sales_item` 的作者定义了类对象可以执行的所有动作。即，`Sales_item` 类定义了创建一个 `Sales_item` 对象时会发生什么事情，以及对 `Sales_item` 对象进行赋值、加法或输入输出运算时会发生什么事情。

一般而言，类的作者决定了类类型对象上可以使用的所有操作。当前，我们所知道的可以在 `Sales_item` 对象上执行的全部操作就是本节所列出的那些操作。

21 读写 `Sales_item`

既然已经知道可以对 `Sales_item` 对象执行哪些操作，我们现在就可以编写使用类的程序了。例如，下面的程序从标准输入读入数据，存入一个 `Sales_item` 对象中，然后将 `Sales_item` 的内容写回到标准输出：

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item book;
    // 读入 ISBN 号、售出的册数以及销售价格
    std::cin >> book;
    // 写入 ISBN、售出的册数、总销售额和平均价格
    std::cout << book << std::endl;
    return 0;
}
```

如果输入：

```
0-201-70353-X 4 24.99
```

则输出为：

```
0-201-70353-X 4 99.96 24.99
```

输入表示我们以每本 24.99 美元的价格售出了 4 册书，而输出告诉我们总售出册数为 4，总销售额为 99.96 美元，而每册书的平均销售价格为 24.99 美元。

此程序以两个 `#include` 指令开始，其中一个使用了新的形式。包含来自标准库的头

文件时，也应该用尖括号（< >）包围头文件名。对于不属于标准库的头文件，则用双引号（" "）包围。

在 main 中我们定义了一个名为 book 的对象，用来保存从标准输入读取出的数据。下一条语句读取数据存入对象中，第三条语句将对象打印到标准输出上并打印一个 endl。

Sales_item 对象的加法

下面是一个更有意思的例子，将两个 Sales_item 对象相加：

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item item1, item2;
    std::cin >> item1 >> item2;           // 读取一对交易记录
    std::cout << item1 + item2 << std::endl; // 打印它们的和
    return 0;
}
```

如果输入如下内容：

```
0-201-78345-X 3 20.00
0-201-78345-X 2 25.00
```

则输出为：

```
0-201-78345-X 5 110 22
```

此程序开始包含了 Sales_item 和 iostream 两个头文件。然后定义了两个 Sales_item 对象来保存销售记录。我们从标准输入读取数据，存入两个对象之中。输出表达式完成加法运算并打印结果。

值得注意的是，此程序看起来与第 5 页的程序非常相似：读取两个输入数据并输出它们的和。造成如此相似的原因是，我们只不过将运算对象从两个整数变为两个 Sales_item 而已，但读取与打印和的运算方式没有发生任何变化。两个程序的另一个不同之处是，“和”的概念是完全不一样的。对于 int，我们计算传统意义上的和——两个数值的算术加法结果。对于 Sales_item 对象，我们用了一个全新的“和”的概念——两个 Sales_item 对象的成员对应相加的结果。

使用文件重定向

当你测试程序时，反复从键盘敲入这些销售记录作为程序的输入，是非常乏味的。大多数操作系统支持文件重定向，这种机制允许我们将标准输入和标准输出与命名文件关联起来：

```
$ addItems <infile >outfile
```

假定 \$ 是操作系统提示符，我们的加法程序已经编译为名为 addItems.exe 的可执行文件（在 UNIX 中是 addItems），则上述命令会从一个名为 infile 的文件读取销售记录，并将输出结果写入到一个名为 outfile 的文件中，两个文件都位于当前目录中。

1.5.1 节练习

练习 1.20: 在网站 <http://www.informit.com/title/0321714113> 上, 第 1 章的代码目录中包含了头文件 `Sales_item.h`。将它拷贝到你自己的工作目录中。用它编写一个程序, 读取一组书籍销售记录, 将每条记录打印到标准输出上。

练习 1.21: 编写程序, 读取两个 ISBN 相同的 `Sales_item` 对象, 输出它们的和。

练习 1.22: 编写程序, 读取多个具有相同 ISBN 的销售记录, 输出所有记录的和。

23

1.5.2 初识成员函数

将两个 `Sales_item` 对象相加的程序首先应该检查两个对象是否具有相同的 ISBN。方法如下:

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item item1, item2;
    std::cin >> item1 >> item2;
    // 首先检查 item1 和 item2 是否表示相同的书
    if (item1.isbn() == item2.isbn()) {
        std::cout << item1 + item2 << std::endl;
        return 0;    // 表示成功
    } else {
        std::cerr << "Data must refer to same ISBN"
                  << std::endl;
        return -1;    // 表示失败
    }
}
```

此程序与上一版本的差别是 `if` 语句及其 `else` 分支。即使不了解这个 `if` 语句的检测条件, 我们也很容易理解这个程序在干什么。如果条件成立, 如上一版本一样, 程序打印计算结果, 并返回 0, 表明成功。如果条件失败, 我们执行跟在 `else` 之后的语句块, 打印一条错误信息, 并返回一个错误标识。

什么是成员函数?

这个 `if` 语句的检测条件

```
item1.isbn() == item2.isbn()
```

调用名为 `isbn` 的**成员函数** (member function)。成员函数是定义为类的一部分的函数, 有时也被称为**方法** (method)。

我们通常以一个类对象的名义来调用成员函数。例如, 上面相等表达式左侧运算对象的第一部分

```
item1.isbn()
```

使用**点运算符** (.) 来表达我们需要“名为 `item1` 的对象的 `isbn` 成员”。点运算符只能用于类类型的对象。其左侧运算对象必须是一个类类型的对象, 右侧运算对象必须是该类型的一个成员名, 运算结果为右侧运算对象指定的成员。

当用点运算符访问一个成员函数时，通常我们是想（效果也确实是）调用该函数。我们使用调用运算符（`()`）来调用一个函数。调用运算符是一对圆括号，里面放置实参（`argument`）列表（可能为空）。成员函数 `isbn` 并不接受参数。因此

```
item1.isbn()
```

调用名为 `item1` 的对象的成员函数 `isbn`，此函数返回 `item1` 中保存的 ISBN 书号。

24

在这个 `if` 条件中，相等运算符的右侧运算对象也是这样执行的——它返回保存在 `item2` 中的 ISBN 书号。如果 ISBN 相同，条件为真，否则为假。

1.5.2 节练习

练习 1.23: 编写程序，读取多条销售记录，并统计每个 ISBN（每本书）有几条销售记录。

练习 1.24: 输入表示多个 ISBN 的多条销售记录来测试上一个程序，每个 ISBN 的记录应该聚在一起。

1.6 书店程序

现在我们已经准备好完成书店程序了。我们需要从一个文件中读取销售记录，生成每本书的销售报告，显示售出册数、总销售额和平均售价。我们假定每个 ISBN 书号的所有销售记录在文件中是聚在一起保存的。

我们的程序会将每个 ISBN 的所有数据合并起来，存入名为 `total` 的变量中。我们使用另一个名为 `trans` 的变量保存读取的每条销售记录。如果 `trans` 和 `total` 指向相同的 ISBN，我们会更新 `total` 的值。否则，我们会打印 `total` 的值，并将其重置为刚刚读取的数据（`trans`）：

```
#include <iostream>
#include "Sales_item.h"
int main()
{
    Sales_item total; // 保存下一条交易记录的变量
    // 读入第一条交易记录，并确保有数据可以处理
    if (std::cin >> total) {
        Sales_item trans; // 保存和的变量
        // 读入并处理剩余交易记录
        while (std::cin >> trans) {
            // 如果我们仍在处理相同的书
            if (total.isbn() == trans.isbn())
                total += trans; // 更新总销售额
            else {
                // 打印前一本书的结果
                std::cout << total << std::endl;
                total = trans; // total 现在表示下一本书的销售额
            }
        }
        std::cout << total << std::endl; // 打印最后一本书的结果
    } else {
```